OpenCV 5 Agentic Vision Sentinel — Technical Report

Competition: OpenCV AI Competition 2026, powered by AWS
Team: Tingqiu Liu (@liutingqiu)
Repository: https://github.com/liutingqiu/opencv5-aws-agentic-vision-sentinel
Runtime measured on: OpenCV 5.0.0, Python 3.13.14, Windows 11, x86-64 CPU (no GPU)

---

1. Problem and users

Fixed-camera sites — substations, loading docks, construction perimeters, remote asset yards — already own the camera but not the attention. A human watching 12 streams misses most events; shipping every frame to a cloud inference service costs more than the guard it replaces and adds a round trip that makes any alert arrive after it mattered.

The users this targets:

- **Site safety officers** who need to know that someone entered a restricted area or has stopped moving inside it.
- **Operations engineers** who want evidence attached to an alert, not a notification that says "motion detected".
- **Small operators** who cannot fund a 24/7 cloud video pipeline and need the analysis to run on hardware they already have.

The design constraint follows from that: **decide at the edge, escalate to the cloud only when a decision was made.**

2. What the system does

Four stages, executed per frame:

1. **OpenCV 5 detection** — SSD-MobileNet-v1 (COCO) exported to ONNX and executed by the OpenCV 5 DNN engine, filtered to the person class with geometric sanity checks.
2. **Classical motion channel** — MOG2 background subtraction plus contour extraction, kept independent of the learned detector so the two signals can be compared.
3. **Spatial decision layer** — a polygon zone, a ground-contact point per detection, greedy frame-to-frame association, and dwell timing. This is where a detection becomes an event.
4. **AWS escalation** — on an event, the annotated frame is written to S3, a structured detail is published to EventBridge, and an alert message is queued in SQS.

A fifth artifact stage writes everything the run produced (`annotated.mp4`, `evidence/*.jpg`, `events.jsonl`, `metrics.json`, `dispatches.json`, `aws_calls.jsonl`) so the numbers in this report, the demo video, and the web page all come from one directory.

3. OpenCV 5 implementation

`cv2.__version__ == 5.0.0`. Two OpenCV 5 specifics shaped the code:

DNN input contract. The 5.0 DNN engine validates the declared graph input instead of coercing it. `ssd_mobilenet_v1_10.onnx` declares a uint8 `image_tensor:0`, so `cv2.dnn.blobFromImage` (float32 NCHW) is rejected with `CV_32FC1 given, CV_8UC1 expected`. The tensor is therefore built as a uint8 NHWC array:

```
tensor = cv2.resize(frame, (300, 300)).astype(np.uint8)[None]
self.net.setInput(tensor)
raw = self.net.forward(self.output_names)   # one graph execution, four outputs
```

Requesting each output with `forward(name)` re-runs the graph and measured **92 ms** instead of roughly **25 ms** per frame. That single change is the difference between a 10 fps budget being met and missed.

Legacy API surface. `cv2.HOGDescriptor` is gone in 5.0, which ruled out the usual pedestrian baseline; MOG2, `findContours` (two-value return), `pointPolygonTest`, `morphologyEx`, and the HSV conversions all behave as expected.

Modules used: `cv2.VideoCapture`, `cv2.dnn`, `cv2.createBackgroundSubtractorMOG2`, `cv2.threshold`, `cv2.dilate`, `cv2.morphologyEx`, `cv2.findContours`, `cv2.contourArea`, `cv2.boundingRect`, `cv2.cvtColor`, `cv2.inRange`, `cv2.bitwise_and/or`, `cv2.fillPoly`, `cv2.polylines`, `cv2.pointPolygonTest`, `cv2.VideoWriter`, `cv2.imwrite`.

4. AWS deployment

One boto3 boundary (`sentinel/aws_dispatch.py`), three services, one code path:

Step	Call	What it carries
Evidence	`s3.put_object`	JPEG of the annotated frame plus anomaly/frame metadata
Event	`events.put_events`	`Source=sentinel.edge.cv5`, `DetailType=SentinelAnomaly`, JSON detail
Alert	`sqs.send_message`	Same detail, with severity and anomaly as message attributes

`--mode aws` uses ambient credentials and an existing bucket/queue (`AwsConfig`). `--mode local` routes the identical calls through moto so the request/response chain — ETag, `EventId`, `MessageId`, MD5 — is exercised and logged without an account. **The recorded run used `local`.** This is stated here, on the project page, and in the video: the cloud boundary is implemented and reproducible, and the identifiers in `runs/demo/dispatches.json` come from a real boto3 round trip, but they were produced by the moto backend, not by an AWS account.

Deploying against a live account is one command after credentials are present:

```
AWS_REGION=us-east-1 AWS_PROFILE=sentinel \
  python -m sentinel.cli --source assets/vtest.avi --out runs/aws --mode aws
```

5. Evaluation

Input. `assets/vtest.avi` — 768x576, 10 fps, 795 frames (79.5 s), a fixed-camera pedestrian scene from the OpenCV sample set. No tuning frames were held out; the clip is the only data used, which is a real limitation of this evaluation and is treated as such in §7.

Hardware. Single-socket x86-64 CPU, no GPU, no OpenVINO/NPU acceleration, single-threaded pipeline loop.

Result of the submitted run:

Metric	Value
Frames analysed	795 / 795 (no drops)
Detections per frame (mean / peak)	4.48 / 9
Frames with at least one person	795 / 795
Inference latency p50	23.31 ms
Inference latency p95	30.62 ms
Inference latency max	34.89 ms
End-to-end frame budget (mean)	31.54 ms
Motion mask coverage (mean)	2.75 % of pixels
Events raised	92 (68 `INTRUSION_DETECTED`, 24 `DWELL_EXCEEDED`)
Alerts dispatched	92 (S3 + EventBridge + SQS each)

Against a 10 fps source (100 ms per frame) one stream consumes about 32 ms of that budget, roughly 3x real-time headroom. Four streams would need 126 ms per 100 ms of wall clock, which does not fit; a single CPU node sustains two streams at 10 fps with margin. That is the capacity number, measured rather than assumed.

Reproduction. `pytest tests -q` → 9 passed in ~7 s. The suite covers real-frame detection, geometric filtering, zone entry/dwell semantics (exactly one alert per track per entry), end-to-end artifact writing, and the cloud dispatch contract (asserting non-empty `MessageId` / `EventId` and `FailedEntryCount == 0`).

6. Failure cases and limitations

These are measured behaviours, not hypotheticals:

1. **Static false positives.** At 300x300 input the SSD cannot separate a distant pedestrian from a bollard or bin. The submitted run still contains detections on static objects at score ~0.42-0.50. A minimum box height of 40 px removed the worst of them; it did not remove all.
2. **No identity persistence.** Association is greedy nearest-ground-point with a 90 px gate. Two people crossing inside the zone can swap tracks, which splits one dwell interval into two and can double-count an alert. A real tracker (ByteTrack/OC-SORT) is the fix; it is not implemented here.
3. **Zone geometry is hand-drawn** in normalised coordinates for this camera. Any other camera needs a new polygon — there is no auto-calibration.

4. **Thermal stage is a colour proxy.** `THERMAL_PROXY_HIGH` fires on warm-hue, high-saturation regions inside the zone. It is not radiometric thermography and it did not fire on this clip, so it carries no measured evidence — it is reported as implemented, not as validated.
5. **The sample clip has no PPE ground truth.** Helmet/vest logic that appeared in the previous description is not implemented, because the evaluation data cannot support it. It is deliberately absent rather than claimed.
6. **No accuracy benchmark.** Precision/recall against a labelled set was not run. The report gives latency, event counts and qualitative inspection of annotated frames; it does not claim a detection F1.

7. Responsible operation

- Frames never leave the device unless an event fires; the S3 write is one JPEG of an already-annotated frame, not a video stream.
- The sample clip is a public OpenCV test asset of an outdoor scene; no live camera, no personal data, no faces collected for this evaluation.
- Alerts carry a track id, a timestamp and an evidence key — enough for a human to check, never a decision taken on a person automatically.
- `events.jsonl` and `dispatches.json` are the audit trail: every alert on the page can be traced to a frame index, an evidence file and a cloud request identifier.
- A production deployment would need retention limits on the evidence bucket and a DPIA for the site; neither is in scope here.

8. What is deliberately not claimed

The earlier version of this submission described an "OpenCV 5 multimodal edge pipeline" whose implementation contained no OpenCV call and no AWS request. That description was wrong and has been replaced. What exists now is narrower and verifiable: OpenCV 5 really decodes and analyses frames, and a real boto3 layer really publishes alerts. Where evidence is missing — accuracy against labels, COOL on Graviton, PPE detection — this report says so instead of asserting it.